

Alius CLI 架构与技术总览

文档状态：中文交付版架构报告

生成日期：2026-06-18

来源范围：.alius/workspace/SPEC.md、.alius/workspace/docs/**、.alius/workspace/ROADMAP.md

适用对象：架构负责人、模块负责人、评审人员、实现人员、项目管理人员

根据 Alius workspace 权威设计文档整理。

1. 文档目的

本文档把当前 Alius CLI 的设计文档整理成一份中文架构报告，方便做 PDF 阅读、评审和任务拆解。它不是新的正式设计契约，也不替代 .alius/workspace/SPEC.md。如果本文档和正式设计文档出现冲突，仍然 以以下顺序为准：

1. .alius/workspace/SPEC.md
2. .alius/workspace/docs/**
3. .alius/workspace/HISTORY.md
4. .alius/workspace/ROADMAP.md，仅作为规划参考

本文档会刻意区分“已实现”“部分接入”“脚手架”“规划中”四类状态。原因是当前项目里有一些模块已经有命令、类型或配置结构，但并不代表 这些能力已经完整接入默认运行路径。尤其是 Agent Team、A2A、MCP、WASM 插件、Workflow、JSON-RPC 等能力，必须看它们是否已经真正进入 默认产品路径、权限路径、事件路径和测试路径。

2. 一句话总结

Alius CLI 是一个本地优先的 Rust Agent Runtime Workspace。它不是一个 简单聊天终端，而是通过 CLI/TUI/JSON-RPC 等产品入口，把请求统一收敛到 Protocol Interface，再进入 Core Runtime，由 Core Runtime 统一管理 会话、运行、事件、模型、工具、权限、存储和扩展系统。

核心架构是：

```
产品入口
-> 协议接口层
-> Core Runtime Manager
-> Core Runtime
-> 运行时子系统
```

默认交互体验是 Plan-driven Agent Runtime Workspace。用户进入 alius 后，默认进入 Ratatui TUI 工作区；只有设置 ALIUS_LEGACY_REPL=1 时，才会进入旧 REPL。

Agent Team 是规划中的多 Agent 协作能力。正确的主体术语是 Agent。本地 Rust CLI 进程是 Agent CLI，它主动通过 WebSocket 连接到 Agent Team Backend。后台只负责注册、在线状态、工作状态、任务租约和 团队事件协调；本地 shell、文件系统、插件、工具执行、确认链路和审计 必须继续留在本地 Agent CLI 与 Core Runtime 内。

3. 状态定义

后续评审和计划都应使用同一套状态词：

- 已实现：代码存在，并已经接入目标产品路径或运行时路径。
- 部分接入：代码存在，但入口、权限、确认、事件、持久化或测试还不完整。
- 脚手架：结构、类型、命令或 UI 已存在，但默认运行路径并不会把它当作 真实能力使用。
- 规划中：设计方向已经明确，但实现仍是后续工作。

不能把“仓库里有代码”直接等同于“产品能力已经完成”。例如，某个命令能 解析，并不代表它已经接入 Core Runtime；某个 UI Tab 能渲染，也不代表 它已经有真实网络事件；某个插件能被发现，也不代表权限、确认、审计和 运行时路径已经完整闭环。

4. 仓库与包结构

当前设计文档描述的主要 Rust workspace 包如下。

4.1 alius-cli

路径：entrypoints/cli

职责：

- CLI 参数解析；
- TUI 工作区启动；
- 旧 REPL 兼容；
- init、config、model、soul、plugin、mcp、workflow 等命令入口；
- 产品层适配；
- 本地交互体验。

CLI 层不应该直接拥有模型供应商内部逻辑、存储结构、工具实现和 Core Runtime 内部流程。默认执行路径应该通过协议桥接进入 Core Runtime。

4.2 jsonrpc

路径：entrypoints/jsonrpc

职责：

- 提供轻量 JSON-RPC 适配器；
- 暴露运行时健康检查、配置读取、模型列表、工具列表、运行启动、运行事件快照、取消和工具确认等接口；
- 作为外部系统接入 Alius 的一条集成路径。

当前 JSON-RPC 的 run_subscribe 设计为快照式读取，不应描述成 server push、连续订阅或长轮询。

4.3 protocol-interface

路径：protocol

职责：

- 定义稳定协议契约；
- 定义协议信封；
- 定义 Origin、CapabilityScope、Capability；
- 定义 CoreRequest、CoreCommand、CoreEvent；
- 定义 ProtocolError；
- 定义 CoreRuntimeApi；
- 提供 Direct Rust API 网关。

4.4 core-runtime

路径：runtime/core

职责：

- Core Runtime 主体；
- Core Runtime Manager 本地门面；
- Session Manager；
- 循环执行模块；
- 事件适配；

- 日志辅助；
- 运行时状态协调；
- 与模型、工具、配置、存储子系统协作。

4.5 runtime-config

路径：runtime/config

职责：

- 项目配置加载；
- 配置视图解析；
- /init 状态机；
- 项目初始化文件创建；
- provider/model/soul/tool/permission/protocol 配置视图；
- 配置迁移辅助。

4.6 runtime-model

路径：runtime/model

职责：

- LLM provider 客户端；
- 模型列表获取；
- 非流式和流式 chat；
- 工具调用帧；
- provider 错误映射；
- Plan/Execute/Review 模型路由。

设计中明确出现的内置 provider 包括：

- bigmodel
- xiaomi_mimo
- deepseek

Google provider 相关能力在设计中要求谨慎，不应在未验证前宣称完整。

4.7 runtime-tools

路径：runtime/tools

职责：

- 原生工具；
- ToolRegistry；
- Shell Gate；
- WASM Host；
- 插件包解析；
- 插件权限；

- MCP 工具注册；
- 工具执行抽象。

4.8 runtime-store

路径：runtime/store

职责：

- 运行时存储；
- 项目本地状态；
- memory/logs 等持久化辅助。

5. 总体架构

Alius 的架构应该理解为分层系统：

产品入口层
 alius CLI
 Ratatui TUI
 JSON-RPC adapter
 未来 Desktop / IDE / SDK / Agent Team / A2A

协议接口层
 ProtocolEnvelope
 CoreRequest
 CoreCommand
 CoreEvent
 Origin
 CapabilityScope
 ProtocolInterface
 ProtocolBridge

Core Runtime 层
 CoreRuntimeManager
 CoreRuntime
 SessionManager
 循环执行模块

运行时子系统
 runtime-config
 runtime-model
 runtime-tools
 runtime-store

最重要的边界规则是：产品入口不能跳过协议层直接调用 Core Runtime 内部 模块。所有产品入口都应该把用户输入或外部请求整理成协议层可识别的请求、命令和事件。

协议层不是业务执行层。它负责校验协议版本、校验 Origin 的能力上限、记录运行上下文、包装事件，然后委托 Core Runtime。模型调用、工具实现、存储结构、TUI 状态都不应该进入协议层。

6. 产品入口

6.1 CLI

CLI 是 Alius 当前最主要的产品入口。它负责：

- 解析命令；
- 应用 locale；
- 加载 settings；

- 分发子命令；
- 启动 TUI 工作区；
- 在需要时启动旧 REPL；
- 暴露初始化、配置、模型、soul、插件、MCP、workflow 等管理入口。

重要命令族包括：

- alius
- alius repl
- alius run
- alius config
- alius version
- alius init
- alius core
- alius soul
- alius plugin
- alius mcp
- alius workflow

设计中的注意事项：

- 一些 root flags 已经定义，但不能默认认为所有路径都完整消费了这些参数。
- extension 管理命令存在，不代表 extension 已经进入默认运行时路径。
- 旧 REPL 只是兼容路径，不应该继续作为主产品模型。
- CLI/TUI 层还有历史显示和兼容状态，但默认执行应持续向协议边界收敛。

6.2 TUI 工作区

TUI 是 Plan-driven Agent Runtime Workspace，不是普通聊天窗口。

默认交互键位：

- Shift+Tab：切换 Plan / Bypass 模式；
- Ctrl+Enter：提交或执行；
- Ctrl+Tab：切换 Conversation / Agent Team；
- Esc：取消、关闭或退出当前交互；
- Ctrl+C / Ctrl+D：退出。

主要区域：

- 顶部状态栏；
- Conversation；
- Plans；
- 交互输入区；
- 底部状态栏；
- 配置和初始化流程；

- Agent Team Tab。

TUI 的长期方向是从 CoreEvent 归约 UI 状态，而不是维护另一套和运行时 重复的状态。Conversation 只展示本地用户和本地运行时流程。Agent Team 必须独立展示真实团队事件，包括方向、发送者、接收者、类型、状态和摘要。

6.3 JSON-RPC

JSON-RPC 是轻量集成入口，当前设计覆盖：

- health_check
- config_read
- model_list
- tool_list
- version
- run_start
- run_subscribe
- run_cancel
- run_confirm_tool

它应该通过 Core Runtime Manager 和 Protocol Interface 进入运行时，不应该绕过协议边界直接访问内部模块。

6.4 npm 分发

npm wrapper 是分发层，不是运行时层。它负责：

- 检测平台和架构；
- 找到对应 native binary；
- 启动 alius；
- 转发参数、stdio 和进程信号。

发布前需要检查 Rust workspace 版本、npm package 版本、平台包版本、tag 和 changelog 是否一致。

7. 协议接口层

协议接口层是产品和运行时之间的契约边界。

核心类型：

- ProtocolEnvelope<T>
- Origin
- CapabilityScope
- Capability
- CoreRequest
- CoreCommand
- CoreEvent
- ProtocolError
- CoreRuntimeApi

ProtocolEnvelope<T> 应携带：

- 协议版本；
- Origin；
- capability scope；
- workspace root；
- session ref；
- run ref；
- trace id；
- payload。

Origin 用于标识请求来源，例如：

- LocalCli
- LocalTui
- EmbeddedSdk
- IdeExtension
- Desktop
- RemoteA2A
- PluginRpc
- JsonRpc
- Test

网关行为：

1. 校验协议版本；
2. 校验 Origin 的 capability ceiling；
3. 委托 CoreRuntimeApi；
4. 存储 run context；
5. 对订阅事件加上原始协议上下文。

CapabilityScope 只是“能力上限”，不是最终授权。最终授权仍必须由运行时、工具权限、Shell Gate、插件权限、任务租约、确认链路和本地策略共同决定。

8. Core Runtime

Core Runtime 是统一执行层。它负责：

- 创建 session；
- 创建 turn；
- 创建 run；
- 执行 Chat / Bypass / Plan 策略；
- 产生 CoreEvent；
- 调用模型运行时；
- 调用工具运行时；

- 读取配置视图；
- 写入或读取 store；
- 处理运行状态；
- 暴露 health、review、log、memory、tool、model 等能力。

CoreRuntimeManager 是本地门面，用于装配本地 runtime 服务，并提供产品层 更容易使用的 API。它不替代 CoreRuntimeApi，也不应该成为新的协议绕行点。

9. Session、Turn、Run、Trace

Session Manager 负责 workspace 级 session 和 run event。

生命周期：

```
create_session
-> create_turn
-> push_event
-> update_run_status
-> get_events 或通过 CoreRuntimeApi subscribe
```

关键标识：

- SessionRef：可恢复的 workspace 上下文；
- TurnRef：一次用户/运行时 turn；
- RunRef：一次执行实例；
- Traceld：贯穿 request、command、event、log、audit；
- RequestId、CommandId、EventId：协议层稳定标识。

当前注意事项：Session Manager 的 session 和 run event 主要在进程内存中 管理。跨进程恢复和持久化能力，需要以 store 模块实际实现为准，不能提前 宣称完整。

10. 事件模型

CoreEvent 是运行时进度和状态的主要表达方式。

重要事件包括：

- RunStarted
- LoopIterationStarted
- ModelDelta
- ModelCompleted
- ToolCallStarted
- ToolCallCompleted
- ConvergenceChecked
- ApprovalRequested
- UserInputRequested
- ErrorRaised
- FinalResult

TUI 应尽量从这些事件归约界面状态。JSON-RPC 和未来远程适配器也应该通过 协议安全的事件包装暴露状态，不应该直接泄露内部运行时结构。

11. 执行模式

11.1 Chat 模式

Chat 模式表示单轮用户对话，但允许受限的工具调用延续。它不是目标导向的 计划执行会话，不要求先生成并确认计划列表。

11.2 Bypass 模式

Bypass 模式直接提交输入执行，不经过本地计划审查。但它仍然必须受运行时 策略、权限、工具确认、Shell Gate 和 workspace 边界约束。

11.3 Plan 模式

Plan 模式是目标导向模式。它应该先讨论目标，形成可执行的 Plan List，然后按计划执行，并保留步骤状态、证据、review 输出和最终结果。

Plan 模式的验收要求更高：

- 必须能表达计划节点；
- 必须能表达步骤状态；
- 工具调用必须可见；
- 高风险工具必须确认；
- 执行结果必须能归档为 evidence；
- final result 必须和计划完成情况对应。

12. 默认运行流程

默认交互流程：

```
alias
-> entrypoints/cli/src/main.rs
-> run_repl(settings)
-> Ratatui workspace, 除非 ALIUS_LEGACY_REPL=1
-> TUI/REPL adapter
-> ProtocolBridge
-> CoreRuntimeManager
-> ProtocolInterface<CoreRuntime>
-> CoreRuntime
-> SessionManager
-> 循环执行模块
-> runtime-model / runtime-tools
```

alias run -p 流程：

```
alias run -p <prompt>
-> CLI dispatch
-> Runtime Manager
-> ProtocolEnvelope<CoreRequest>
-> Core Runtime
-> 执行循环
-> 输出最终结果
```

Plan 执行流程：

用户目标

- > Plan 模式交互
- > 计划草拟与审查
- > 用户接受计划
- > runtime 执行
- > model/tool events
- > step evidence
- > final result

工具确认流程：

- 请求工具调用
- > preview_confirmation 或 Shell Gate 风险判断
- > ToolConfirmationRequired
- > SessionManager 进入等待状态
- > 用户 approve / deny / cancel / timeout
- > 工具执行或 fail closed
- > audit
- > CoreEvent

13. 配置系统

项目配置根目录：

`.alius/config/`

项目配置文件：

- `config.toml`
- `providers.toml`
- `model.toml`
- `soul.toml`
- `tools.toml`
- `permissions.toml`
- `protocol.toml`

用户级 MCP 配置：

- `~/.alius/mcp/servers.toml`

历史 MCP 配置引用：

- `.alius/config/mcp.json`

Config Manager 负责：

- 查找项目根目录；
- 加载项目配置；
- 构建 ProjectConfigSnapshot；
- 解析 RuntimeConfigView；
- 提供 provider/tool/permission/protocol/soul/logging/session 视图；
- 支持项目初始化默认值；
- 支持可恢复 /init 状态；
- 支持已实现范围内的配置迁移。

本地模型库存放在 `.alius/config/providers.toml` 的 `[[model_library.models]]` 下。模型分配存放在 `.alius/config/model.toml`，包括：

- Plan Model ;
- Execute Model ;
- Review Model。

/model 流程负责管理模型库存。/config 流程只应从启用的模型库存条目 里分配 Plan/Execute/Review , 不应在分配阶段要求手动输入模型名、Base URL 或 API Key。

14. 模型运行时

模型运行时负责 provider 和 LLM 请求。

核心职责：

- OpenAI-compatible provider ;
- BigModel ;
- Custom provider ;
- Anthropic native ;
- 模型列表获取 ;
- 流式和非流式 chat ;
- 工具调用帧 ;
- provider 错误映射 ;
- Plan/Execute/Review 角色路由。

DeepSeek 是当前设计中需要重点覆盖的默认 provider 之一。CI 中的 provider smoke test 应验证配置加载、环境变量凭据、transport、response shape、streaming 兼容性和错误映射，但不应断言自然语言文本的精确内容。

15. 工具、ToolRegistry 与 Shell Gate

所有工具应收敛到统一 AliusTool 抽象和 ToolRegistry 注册模型。

工具来源：

- 原生工具 ;
- Rust WASM module plugin tools ;
- MCP tools ;
- workflow tool steps。

原生工具包括：

- shell
- read_file
- write_file
- list_dir
- edit_file

Shell Gate 必须分析：

- 命令 ;

- 命令参数；
- 原始路径；
- redirection；
- 当前工作目录；
- Origin；
- workspace root；
- risk；
- scope。

越过 workspace 边界的行为应 hard deny，例如：

- workspace 外的绝对路径；
- ../ 逃逸；
- 输出重定向到 workspace 外；
- --output=tmp/out 这类输出参数指向 workspace 外。

高风险但仍在 workspace 内的操作可以进入确认流程。workspace 外的副作用 不能通过确认变成允许，应该 fail closed。

ToolContext 应携带：

- workspace；
- session；
- working directory；
- mode；
- trace context。

16. WASM 插件系统

WASM 插件是 extension system 的一部分。

相关能力包括：

- CLI plugin 管理；
- plugin manifest；
- extension registry；
- runtime tool registration；
- WASM host imports；
- manifest permissions；
- resolved permission matcher；
- host audit sink。

官方扩展现在应随主仓库放在 extensions/ 下。extensions/registry.toml 描述官方 soul 和 WASM 插件。alius soul update 应优先从仓库内置 extensions/souls/ 同步，不需要访问旧远程仓库。

WASM host imports 包括：

- read_file

- write_file
- list_dir
- env_get
- shell
- fetch

每个 host import 应走同一条链路：

```
解析 WASM memory JSON
-> 权限 matcher 检查
-> domain security primitive
-> audit log
-> execute 或 return denial
```

安全不变量：

- 不记录文件内容；
- 不记录环境变量值；
- 不记录 shell stdout/stderr；
- 敏感参数必须脱敏；
- 允许和拒绝都要审计；
- audit sink 失败不改变 allow/deny 决策。

仍需追踪的成熟度问题：

- host audit 尚未完整进入 per-session trace review；
- workflow 中需要确认的工具步骤缺少完整交互确认通道时必须 fail closed；
- ABI、sandbox、permission hardening 需要最终安全审查后才能做生产级声明。

17. MCP 集成

MCP 是 extension system 的一部分。

配置位置：

- project switch：.alius/config/tools.toml
- user server declarations：~/.alius/mcp/servers.toml
- legacy path：.alius/config/mcp.json

MCP auto-init 要求以下开关同时满足：

- registry.mcp_tools = true
- mcp.load_on_workspace_start = true
- mcp.register_as_tools = true

当前设计状态：

- CLI 管理能力存在；
- server listing、start、tool listing 行为存在；
- 配置满足时 MCP tools 可以进入共享 ToolRegistry；
- 初始化在后台运行，不应阻塞 runtime startup；

- native/WASM tools 优先，MCP 重名工具应跳过。

MCP 工具如果能产生本地副作用，必须被纳入统一权限、安全和审计模型。

18. Workflow Runtime

Workflow 命令位于 `entrypoints/cli/src/workflow/`。

当前设计状态：

- CLI command surface 和 parsing 存在；
- runtime integration 通过 handle trait 表达；
- prompt/tool/condition step 通过该 trait 执行；
- condition 支持 contains、success、failed；
- test stub 只应保留给单元测试；
- workflow run 应使用真实 CoreRuntimeManager 和 ToolRegistry；
- prompt step 调用 LLM provider；
- tool step 走真实 native/WASM/MCP tool path。

重要缺口：

- HTTP step 当前使用 `request::Client` 直接请求，尚未纳入统一 permission model。

因此 workflow 的网络行为不能视为安全治理完整。它还需要统一 timeout、allowlist、audit、secret redaction 和 policy enforcement。

19. Soul 与 Agent Card

Soul 和 Agent Card 用于描述本地 Agent 身份、角色、能力提示和项目行为。

相关路径：

- `entrypoints/cli/src/formula/`
- `runtime/config/src/agent_card.rs`
- `runtime/config/src/soul.rs`
- `runtime/config/src/soul_source.rs`
- `extensions/souls/`
- `.alius/config/soul.toml`

官方 souls 应放在主仓库 `extensions/souls/`。推荐同步路径是 `alius soul update`。旧 `alius core update` 可以保留兼容，但不应作为官方扩展路径。

20. Agent Team 架构

Agent Team 是规划中的多 Agent 协作能力。术语必须统一：

- Agent：本地协作身份，包含 Soul、role、capabilities 和 workspace context。
- Agent CLI：本地 Rust CLI 进程，代表 Agent 连接团队后台并执行本地任务。
- Agent Team Backend：服务端协调 API，当前设计计划基于 FastAPI。

- Agent Connection : 一个长期 WebSocket 连接。
- Agent Presence : 连接级状态。
- Agent Work Status : 执行级状态。
- Agent Task Lease : 后台签发的任务执行租约。

第一阶段推荐 transport :

Agent CLI --wss--> Agent Team Backend

Agent CLI 必须主动向后台发起 outbound WebSocket 连接。它不应该打开本地 inbound 端口让后台回调。后台可以和 REST API 共用 HTTPS 服务端口，例如：

GET /api/agent/ws

生产环境使用 wss:// 走 443。开发环境可以使用：

ws://localhost:<port>/api/agent/ws

推荐 Rust client stack :

- tokio
- tokio-tungstenite
- futures
- serde
- serde_json
- uuid
- tokio-util cancellation tokens
- tracing
- reqwest

FastAPI 后台可以同时提供：

```
GET /api/team/agents
POST /api/team/tasks
GET /api/team/tasks/{task_id}
```

但后台只能协调任务和事件，不能直接执行本地 shell、文件系统、插件或工具 操作。本地执行权必须留在 Agent CLI 和 Core Runtime。

21. Agent Team 协议

WebSocket upgrade 应携带认证和协议选择：

```
GET /api/agent/ws
Authorization: Bearer <agent-token>
X-Agent-Id: <agent-id>
X-Workspace-Id: <workspace-id>
Sec-WebSocket-Protocol: alius-team.v1
```

连接建立后，第一个 application message 仍必须是 Agent 注册消息。原因是 连接级认证不足以表达当前 Agent 身份、workspace、协议版本、capabilities 和 replay 位置。

注册消息应包含：

- protocol version ;
- message id ;
- agent id ;

- instance id ;
- workspace id ;
- soul ;
- role ;
- requested capabilities ;
- last seen sequence。

后台返回 granted 和 denied capabilities。Agent CLI 自声明的 capabilities 只是请求，不是最终授权。

每条 Agent Team message 应包含：

- protocol version ;
- message id ;
- connection id ;
- agent id ;
- team id ;
- workspace id ;
- trace id ;
- connection-scoped sequence ;
- timestamp ;
- type ;
- payload。

Agent CLI 到后台的消息类型：

- RegisterAgent
- Heartbeat
- StatusUpdate
- RunEvent
- TaskAccepted
- TaskRejected
- TaskProgress
- TaskResult
- ConfirmationRequired
- ErrorReport

后台到 Agent CLI 的消息类型：

- RegisterAck
- TaskOffer
- CancelRun
- PauseRun
- ResumeRun
- ConfirmationDecision

- SyncRequest
- ConfigUpdate
- ShutdownNotice
- Error

未知消息类型应返回结构化错误，不应无条件断开连接。只有协议违规或安全策略违规时才应关闭 socket。

22. Agent Presence、Work Status 与 Task Lease

Presence 是连接级状态：

- connecting；
- online；
- syncing；
- degraded；
- reconnecting；
- offline。

Work Status 是执行级状态：

- idle；
- planning；
- running；
- streaming；
- waiting for approval；
- running tool；
- reviewing；
- blocked；
- completed；
- failed；
- cancelled。

两者必须分开。Agent 可以 online 但正在等待用户确认；Agent 也可以 degraded，但后台仍需要根据任务租约做状态协调。

Heartbeat 建议：

- Agent CLI 连接期间每 5 秒发送 heartbeat；
- 15 秒无 heartbeat，后台标记 degraded；
- 30 到 45 秒无 heartbeat 或 socket closed，后台标记 offline。

任务分配必须基于 lease：

```
TaskOffer
-> TaskAccepted(lease_id)
-> RunEvent*
-> TaskResult
```

规则：

- 后台 task offer 必须包含 task id 和 lease TTL ；
- Agent CLI 必须先 TaskAccepted 再执行 ；
- Agent CLI 工作期间必须 renew lease ；
- 后台不能把同一个 active task 分配给多个 Agent ，除非任务显式 parallel ；
- Agent 断连超过 lease TTL 后 ，后台才能标记 lost、failed 或重新分配。

Reconnect 必须生成新的 connection id。旧 connection id 不能继续代表权限。Agent CLI reconnect 时应携带 last_seen_seq 和当前本地 run 状态。

23. Agent Team 权限模型

WebSocket 本身不提供业务权限。权限必须由 application protocol 和本地 运行时共同实现。

规则：

- WebSocket upgrade 要认证 ；
- upgrade 后必须注册并授权 Agent ；
- 自声明 capabilities 只是请求 ；
- 每条 control message 都必须校验 agent id、team id、workspace id、granted capabilities 和 active lease ；
- 后台不能静默批准本地高风险操作 ；
- 本地工具、shell、文件系统、插件和确认策略留在 Agent CLI 与 Core Runtime ；
- 任务分配、取消、确认决策、失败和权限拒绝都要审计。

推荐 capability 名称：

- join_team
- receive_task
- delegate_task
- use_model
- read_workspace
- write_workspace
- use_tools
- use_shell
- use_mcp
- read_memory
- write_memory
- approve_remote_confirmation

approve_remote_confirmation 默认必须拒绝。即使未来支持远程确认，也只能转发用户在后台做出的明确决定，不能绕过本地 runtime confirmation gate。

24. 数据与状态布局

项目本地状态：

.alius/

```

config/
  config.toml
  providers.toml
  model.toml
  soul.toml
  tools.toml
  permissions.toml
  protocol.toml
memory/
  communications/sessions/
  logs/
  design/
workspace/
  SPEC.md
  docs/
  HISTORY.md
  ROADMAP.md

```

用户级状态：

```

~/.alius/
  mcp/
  servers.toml

```

边界说明：

- `.alius/workspace/` 是当前权威文档区；
- `.alius/memory/` 是运行时 memory 和 logs；
- `.alius/memory/design/` 是历史设计输入，不应覆盖当前 workspace docs。

25. 安全架构

Alius 的安全应该是多层防线，不依赖某一个单点。

25.1 协议能力上限

CapabilityScope 是产品入口声明的能力上限，不是最终授权。具体动作仍需要 runtime、工具权限、Shell Gate、插件权限、任务租约和确认链路共同判断。

25.2 Tool 与 Shell Safety

shell 和文件系统操作必须检查：

- command；
- args；
- redirection；
- output path；
- cwd；
- workspace root；
- risk；
- origin；
- mode。

workspace 逃逸必须 hard deny。workspace 内高风险操作可以要求确认。

25.3 WASM 插件权限

插件通过 manifest 声明 filesystem、network、shell、env 权限。运行时每次 host import 都要检查 manifest、检查具体参数、通过共享安全原语、写 audit，然后允许或拒绝。

25.4 Workflow 网络安全

Workflow HTTP step 当前未统一纳入 permission model，因此不能视为安全完整。后续需要加入 allowlist、timeout、audit、secret redaction 和 policy。

25.5 Agent Team 安全

后台协调任务，但本地执行权仍在本地。远程任务和控制命令不能绕过：

- local capability；
- active lease；
- Shell Gate；
- workspace boundary；
- plugin permissions；
- user confirmation；
- audit logging。

25.6 Secret 与 CI 日志

Provider network tests 不能打印 API key、Authorization header、secret query params、包含 secret 的生成配置文件或完整 env dump。CI logs 和 artifacts 必须脱敏。

26. 测试与 CI

设计文档要求测试体系尽可能完整，至少覆盖：

- parser 和 unit tests；
- CLI command functional tests；
- Core Runtime；
- Protocol Interface；
- Session Manager；
- 循环执行模块；
- tools；
- plugin；
- MCP；
- workflow；
- JSON-RPC；
- TUI state-machine；
- local mock HTTP；
- local fixture MCP；
- shell/filesystem/network/env/confirmation/permission-denial；
- release build smoke。

TUI 重点不应依赖人工操作，而应通过 TuiTestHarness 覆盖状态机：

- 模式切换；
- 输入提交；
- Plan 节点状态；
- Conversation 与 Agent Team 分离；
- init/config/model flow；
- error display；
- resize 或 responsive state；
- key binding。

测试辅助代码必须隔离在 release binary 之外：

```
#[cfg(any(test, feature = "testing"))]
pub mod testing;
```

测试命令：

```
cargo test --workspace --features testing --locked
```

release build 命令：

```
cargo build -p alius-cli --bin alius --release --locked
```

release build 禁止：

```
cargo build --release --all-features
cargo build --release --features testing
```

CI 应先跑测试和覆盖率，再做 build。测试报告和覆盖率报告保留在 CI 原生 logs、job summary 和 artifacts 内。不要加入外部测试报表服务。

Selected provider smoke tests 可以在 CI secrets 配置后开启。它们不是单独的产品模式，只是验证选定 provider 的配置、凭据、transport、response parsing 和 streaming compatibility。DeepSeek 作为默认 provider 时，应作为代表性配置流程被覆盖。

27. 当前重点缺口

从设计角度，后续仍需重点跟踪：

- CLI root flags 是否在所有 dispatch path 里完整消费；
- TUI 是否真正从 CoreEvent 归约状态；
- Plan 模式下工具执行、确认、证据和 review 是否全链路完整；
- 工具权限是否覆盖所有工具来源；
- Shell Gate 是否检查 args、redirection 和输出路径；
- Workflow HTTP step 是否进入统一 permission model；
- WASM plugin ABI、sandbox、permission hardening 是否完成最终安全评审；
- WASM host audit 是否进入 per-session trace review；
- MCP 是否在配置满足时正确注册到 ToolRegistry；
- MCP duplicate tool name 是否稳定跳过；
- Agent Team live traffic 是否真正接入；
- Agent CLI WebSocket connector 是否完成；

- Agent presence、work status、task lease、reconnect、replay 是否完成；
- Agent Team 事件是否真实填充 TUI Agent Team state；
- JSON-RPC subscribe 是否仍是 snapshot，是否需要真正 event stream；
- session persistence 和跨进程恢复是否完成；
- Google provider 是否经过验证。

28. 后续评审清单

每次 review 新功能时，应至少检查：

1. 产品入口是否通过 Protocol Interface 进入运行时；
2. 是否保留 trace id；
3. 是否区分本地 Conversation 和 Agent Team traffic；
4. 是否所有本地副作用都经过安全边界；
5. Shell Gate 是否检查 command、args、paths、redirection 和 cwd；
6. 权限不确定时是否 fail closed；
7. 测试是否可重复；
8. 测试辅助代码是否不会进入 release binary；
9. 文档是否准确区分已实现、部分接入、脚手架、规划中；
10. Agent Team 是否在 WebSocket、registration、presence、lease、reconnect、event mapping、TUI population 都完成测试前避免过度声明。

29. 开发路径建议

如果目标是让核心目标达到 100% 完成，应按以下顺序推进。

阶段一：CLI 与 TUI 基础闭环

目标：

- CLI 所有命令族有 functional tests；
- root flags 在所有路径行为明确；
- TUI state-machine 有 TestKit；
- TUI 不依赖人工操作才能验证核心状态。

验收：

- CLI 命令族测试全覆盖；
- TUI Plan/Bypass、Conversation/Agent Team、init/config/model flow 有测试；
- release build 不包含 testing helper。

阶段二：Protocol 与 Core Runtime 闭环

目标：

- 所有产品入口统一进入协议边界；

- CoreRequest/CoreCommand/CoreEvent 行为稳定；
- Session/Run/Trace 在所有路径贯穿。

验收：

- Protocol Interface 测试覆盖 capability ceiling；
- Core Runtime 事件测试覆盖 start、tool、approval、final result；
- trace id 可从 request 追到 event 和 audit。

阶段三：Tool/Shell Gate/权限闭环

目标：

- 所有工具来源进入统一 ToolRegistry；
- Shell Gate 检查 command args 和 redirection；
- workspace 外副作用 hard deny；
- 高风险 workspace 内操作进入 confirmation。

验收：

- /etc/passwd、../outside、--output=/tmp/out、> /tmp/out、2>/tmp/err 等测试覆盖；
- native/WASM/MCP/workflow tool path 权限一致；
- denial 和 confirmation 有 audit。

阶段四：WASM/MCP/Workflow/JSON-RPC 成熟化

目标：

- WASM plugin 权限和 audit 完整；
- MCP 注册和重名处理稳定；
- workflow HTTP 纳入 permission model；
- JSON-RPC run event 能力按设计完成。

验收：

- WASM host imports 全覆盖测试；
- MCP fixture server 测试；
- workflow prompt/tool/condition/http 测试；
- JSON-RPC run_start、run_subscribe、run_cancel、run_confirm_tool 测试。

阶段五：Agent Team

目标：

- Agent CLI outbound WebSocket；
- FastAPI backend protocol compatibility；
- register/ack；
- heartbeat；
- presence；

- work status ;
- task lease ;
- reconnect/replay ;
- TUI Agent Team state population。

验收：

- handshake rejection ;
- successful registration ;
- heartbeat degraded/offline ;
- unauthorized command rejection ;
- lease acceptance and renewal ;
- reconnect with new connection id ;
- CoreEvent 到 Agent Team event mapping ;
- TUI 只展示真实 Agent Team event。

30. 参考来源

本文档基于以下设计文件整理：

- .alius/workspace/README.md
- .alius/workspace/SPEC.md
- .alius/workspace/ROADMAP.md
- .alius/workspace/docs/00-reading-path.md
- .alius/workspace/docs/01-current-state.md
- .alius/workspace/docs/overview/architecture.md
- .alius/workspace/docs/overview/data-flow.md
- .alius/workspace/docs/overview/diagrams.md
- .alius/workspace/docs/overview/runtime-flow.md
- .alius/workspace/docs/overview/implementation-gaps.md
- .alius/workspace/docs/interfaces/protocol-interface.md
- .alius/workspace/docs/interfaces/core-runtime-api.md
- .alius/workspace/docs/interfaces/events-and-tracing.md
- .alius/workspace/docs/interfaces/config-schema.md
- .alius/workspace/docs/products/cli.md
- .alius/workspace/docs/products/tui-workspace.md
- .alius/workspace/docs/products/jsonrpc.md
- .alius/workspace/docs/products/npm-distribution.md
- .alius/workspace/docs/modules/cli-entypoint.md
- .alius/workspace/docs/modules/protocol.md
- .alius/workspace/docs/modules/core-runtime.md

- [.alius/workspace/docs/modules/session-manager.md](#)
- [.alius/workspace/docs/modules/loop-engine.md](#)
- [.alius/workspace/docs/modules/config-manager.md](#)
- [.alius/workspace/docs/modules/model-runtime.md](#)
- [.alius/workspace/docs/modules/memory-store.md](#)
- [.alius/workspace/docs/modules/tools-and-shell-gate.md](#)
- [.alius/workspace/docs/modules/extensions.md](#)
- [.alius/workspace/docs/modules/plugin-permissions.md](#)
- [.alius/workspace/docs/modules/agent-team.md](#)
- [.alius/workspace/docs/standards/validation.md](#)
- [.alius/workspace/docs/standards/documentation-maintenance.md](#)
- [.alius/workspace/docs/terms/GLOSSARY.md](#)